Introduction to Penetration Testing with Kali Linux

Student Lab Handout | One Kali Linux machine only | About 110–140 minutes | Beginner

Student name: _____	**ID:** _____
Section: _____	**Date:** _____

How to use this handout. Every command you must run is shown in a dark-labelled box. Type (or copy) the commands exactly as shown, one box at a time, in the terminal named in the box label. Lines starting with # are comments: do not type them. Record only your **real** results in the evidence boxes.

Lab overview

In this lab, Kali Linux is both the testing machine and the target. Your instructor has prepared a deliberately misconfigured local website. You will use only your own Kali installation, Firefox, `ss` and Nmap to investigate `127.0.0.1:8000`, discover an exposed file **without being told its name**, document evidence, fix the weakness and retest.

Component	Configuration
Operating system	Kali Linux (physical install or a single VM)
Attacker	Kali Linux terminal on the same machine
Target	Local Python HTTP server on 127.0.0.1:8000
Tools	Python 3, Nmap, Firefox; optional ss and curl
Network requirements	No target network. Installing missing tools may require package access.
Authorization boundary	**ONLY 127.0.0.1 and the lab directory ~/pentest-lab.** Never scan or test any other address.

Learning outcomes

- Explain penetration testing, scope, authorization, vulnerabilities, risks, reconnaissance and remediation.
- Differentiate an IP address, a port, a protocol and a running service.
- Create and verify a local-only HTTP test environment.
- Use ss, Nmap and Firefox to collect reproducible evidence.
- Recognize and explain sensitive-file exposure without accessing any third-party system.
- Apply a safe corrective action and verify the result with a repeat test.
- Write a clear technical finding with evidence, impact, recommendation and retest result.

Part A — Theory and background (15–20 minutes)

1. Essential networking terminology

- **IP address:** Numerical network identifier. The IPv4 loopback address 127.0.0.1 always refers to the same computer running the command.
- **Localhost:** Hostname that usually resolves to a loopback address. This lab uses explicit 127.0.0.1 to avoid IPv4/IPv6 confusion.
- **Port:** An endpoint number used with a transport protocol. This lab uses TCP port 8000.
- **TCP:** Connection-oriented transport protocol used by the local HTTP server.
- **HTTP:** Application protocol for requests and responses between a web client and a web server.
- **HTTP status 200:** The request succeeded. It does not mean the content is safe to disclose.
- **HTTP status 404:** The requested resource was not found; expected after the exposed file is moved.
- **Header:** HTTP response metadata; Server and Content-Type headers may describe service behavior.

2. Tools and security interpretation

- **Terminal:** Runs commands and displays standard output and errors.
- **Python http.server:** Minimal development/test file server. Not suitable for production hosting; it serves files from its specified directory.
- **ss:** Displays socket information, including local listening TCP ports.
- **Nmap:** Network scanner. `-p` selects ports; `-sV` sends probes to identify the actual service.
- **curl (optional):** Command-line HTTP client; lets you repeat a browser request from the terminal.
- **Firefox:** Web browser used to request test pages and, with Developer Tools, inspect HTTP status codes and response headers.
- **Browser Developer Tools:** The Network panel shows requests, status codes, response headers and content returned by the local web server.

Important distinction: Nmap may display a service name based only on the port number, whereas `-sV` actively tries to identify the service. Neither a service banner nor an HTTP Server header is proof of a vulnerability. A publicly retrievable file containing sensitive information **is** direct evidence of an exposure.

3. Threat scenario and severity

Scenario: An administrator accidentally stores an internal text file in the public website directory. You are not given the name or contents of that file. A real deployment could disclose confidential data; this exercise uses fictional practice material only.

Part B — Guided practical activities (60–80 minutes)

Before starting — terminal arrangement

Open **two** Terminal windows or tabs and label them:

- **Terminal A** — runs the web server continuously (Activity 3). Do not close it or press Ctrl+C until Activity 11.
- **Terminal B** — runs every inspection, scan and remediation command.

Unless a box says otherwise, run each command in the box in order, one line at a time.

Activity 1: Check the Kali environment (8 min)

Purpose: Confirm the tools exist before starting.

TERMINAL B

```
whoami
pwd
python3 --version
nmap --version
firefox --version
curl --version
```

What the command does: `whoami` shows the active user; `pwd` shows the current directory; `--version` shows whether a tool is installed and which version.

What you should observe: Version information for Python 3, Nmap, Firefox and curl. If a command says *command not found*, install it with the commands below (only if your instructor allows package access).

Only if a tool is missing (example for Nmap and curl):

TERMINAL B

```
sudo apt update
sudo apt install -y nmap curl
```

Check your understanding: Which command shows the logged-in user?

Activity 2: Enter the instructor-provided website directory (5 min)

Purpose: Start from the directory your instructor prepared, without revealing the exposed file's name.

TERMINAL B

```
cd ~/pentest-lab
pwd
test -d ~/pentest-lab && echo "Lab directory found"
```

What the command does: `cd` enters the practice directory; `pwd` confirms where you are; `test -d` prints a message only if the directory exists.

> **Do NOT list the files** (no `ls`) and do not open the folder in a file manager yet. Finding the web-accessible files is part of the investigation.

What you should observe: The working directory ends with `/pentest-lab` and the message *Lab directory found*. If not, ask your instructor to prepare the lab directory.

Check your understanding: Why should the target content be prepared without telling you which file has the weakness?

Activity 3: Start the target HTTP service — Terminal A (7 min)

Purpose: Launch the local-only web server and keep it running.

TERMINAL A

```
cd ~/pentest-lab
python3 -m http.server 8000 --bind 127.0.0.1
```

What the command does: `python3 -m http.server` starts a basic file server; `8000` selects the listening TCP port; `--bind 127.0.0.1` stops it from listening on any external network interface.

What you should observe: A message similar to *Serving HTTP on 127.0.0.1 port 8000*. The terminal stays busy while the server runs. **Leave it open.** Press Ctrl+C only in Activity 11.

Check your understanding: What is the difference between port 8000 and 127.0.0.1?

Activity 4: Verify the website in Firefox — Terminal B (6 min)

Purpose: Make a baseline browser request before scanning.

Step 1. Open Firefox from Terminal B (the `&` keeps the terminal free):

TERMINAL B

```
firefox http://127.0.0.1:8000/ &
```

Step 2. In Firefox, record what the server displays. Do not assume a normal landing page will appear.

Step 3. Press **Ctrl+Shift+E** (Developer Tools → Network), then **Ctrl+R** to reload. Select the request for `/` and note its HTTP status and response headers.

Optional terminal check of the same request:

TERMINAL B

```
curl -i http://127.0.0.1:8000/
```

What the command does: Firefox (or `curl -i`) sends an HTTP GET request to your local server. The Network panel, or the top of the curl output, shows the status code and headers.

What you should observe: A page headed *Directory listing for /* with clickable filenames. The request to `/` normally returns **HTTP 200**.

Check your understanding: What caused the directory listing to appear, and what kind of information does it reveal?

Activity 5: Confirm the listening socket (6 min)

Purpose: See which interface and port host the test service.

TERMINAL B

```
ss -ltn | grep ":8000"
```

What the command does: `ss` shows sockets; `-l` listening only; `-t` TCP; `-n` numeric addresses; `grep` keeps only lines containing :8000.

What you should observe: A LISTEN line with a local address similar to `127.0.0.1:8000`.

Check your understanding: Would binding to 0.0.0.0 have the same exposure boundary? Why not?

Activity 6: Scan the chosen port with Nmap (8 min)

Purpose: Detect the intentionally running TCP service.

TERMINAL B

```
nmap -p 8000 127.0.0.1
```

What the command does: `nmap` scans the given host; `-p 8000` scans only port 8000; 127.0.0.1 is the authorized loopback target.

What you should observe: Typically `8000/tcp open http-alt`. The service name is a guess based on the port number and may differ.

Check your understanding: Does “open” prove a security vulnerability?

Activity 7: Identify the service version (8 min)

Purpose: Distinguish basic port labelling from active service detection.

TERMINAL B

```
nmap -sV -p 8000 127.0.0.1
```

What the command does: `-sV` enables service/version detection; `-p 8000` keeps the scan limited to the lab port.

What you should observe: Possible identification of an HTTP service such as Python http.server. Exact details vary with tool and Python version.

Check your understanding: How does -sV change the type of evidence compared with Activity 6?

Activity 8: Inspect HTTP response headers (8 min)

Purpose: Record the HTTP metadata returned by the local service.

In Firefox: open `http://127.0.0.1:8000/`, press Ctrl+Shift+E, reload (Ctrl+R), select the request for `/`, choose **Headers**, and inspect Status, Server, Date and Content-Type.

Same information from the terminal (the `-I` option requests headers only):

TERMINAL B

```
curl -I http://127.0.0.1:8000/
```

What you should observe: Status 200 plus response headers such as Server, Date and Content-Type. Values depend on your Python installation.

Check your understanding: Does a Server header by itself demonstrate compromise?

Activity 9: Investigate the directory listing and validate an exposure (12 min)

Purpose: Independently discover whether any browser-accessible file reveals information that should not be public. The filename is intentionally withheld.

Step 1 — Browser. In Firefox open `http://127.0.0.1:8000/`. Study the directory listing. Open each listed file in a new tab and read its contents. Identify the one file that exposes internal information. Write down its exact filename and URL.

Step 2 — Confirm the status. With Developer Tools → Network open (Ctrl+Shift+E), reload the suspicious file's URL, select its request and record the status code.

Step 3 — Optional terminal view of the same evidence. First list the names on the page, then request your file (replace `FILENAME` with the exact name you found):

TERMINAL B

```
 # show the directory listing page as text (look for the href="..."
entries)
 curl -s http://127.0.0.1:8000/ | grep href
 # store the filename you found in a variable (replace the text inside the
quotes)
 FILE="FILENAME"
 # show the HTTP status line and headers for your file
 curl -I http://127.0.0.1:8000/"$FILE"
 # show only the HTTP status code
 curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/"$FILE"
 # display the file's contents
 curl -s http://127.0.0.1:8000/"$FILE"
```

What you should observe: A directory listing of locally served files: one harmless page and one text file containing clearly fictional account information. Requesting the sensitive file returns **HTTP 200**.

Check your understanding: How did you identify the suspicious file without knowing its name beforehand? Which security property is affected?

Evidence to record

Record the actual filename, full loopback URL, HTTP status, relevant contents and a Firefox screenshot. Write only your **real** observations; do not copy example output.

Item	Your observation
Exact filename	
Full URL	
HTTP status	
Why this must not be public	

Activity 10: Remediate the discovered exposure and retest (10 min)

Purpose: Move the file YOU discovered out of the served directory, then prove the original URL no longer returns it.

Step 1 — Move the file. Reuse the `FILE` variable from Activity 9. If you opened a new terminal, set it again with the exact filename (no URL):

TERMINAL B

```
FILE="FILENAME"
mv -- ~/pentest-lab/"$FILE" ~/pentest-lab-private-file.txt
```

Step 2 — Confirm the file is gone from the served directory (you may use `ls` now):

TERMINAL B

```
ls -la ~/pentest-lab
```

Step 3 — Retest. In Firefox reload the original suspicious URL with Developer Tools → Network open and record the status. Terminal equivalent:

TERMINAL B

```
curl -I http://127.0.0.1:8000/"$FILE"
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/"$FILE"
```

Step 4 — Optional. Reload `http://127.0.0.1:8000/` in Firefox and confirm the filename is no longer listed. Terminal equivalent:

TERMINAL B

```
curl -s http://127.0.0.1:8000/ | grep href
```

What the command does: `mv` moves the file outside the public directory. The retest uses the same URL that proved the exposure.

What you should observe: The original file URL now returns **HTTP 404** and the file is absent from `ls` and from the directory listing. If it is still reachable, check that you used the correct filename and the correct website folder.

Check your understanding: Why is retesting essential after remediation?

Optional extensions (do before Activity 11, or ask your instructor)

Extension A — create and remove a harmless file. Do this only after the main investigation, while the server is still running:

TERMINAL B

```
echo "harmless practice note" > ~/pentest-lab/notes.txt
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/notes.txt
mv -- ~/pentest-lab/notes.txt ~/notes-private.txt
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/notes.txt
```

Expected: first command prints `200`, second prints `404`.

Extension B — use a different port. Stop the server in Terminal A with Ctrl+C, then restart it on port 8081 and scan **only** that local port:

TERMINAL A

```
cd ~/pentest-lab
python3 -m http.server 8081 --bind 127.0.0.1
```

TERMINAL B

```
ss -ltn | grep ":8081"
nmap -sV -p 8081 127.0.0.1
```

Press Ctrl+C in Terminal A when finished.

Activity 11: Stop and clean up (5 min)

Purpose: Shut down the test server and confirm the port no longer accepts connections.

Step 1. Click Terminal A and press **Ctrl+C** to stop the server.

Step 2. In Terminal B run:

TERMINAL B

```
ss -ltn | grep ":8000" || echo "Port 8000 is not listening"
nmap -p 8000 127.0.0.1
```

What the command does: The `||` operator prints the message only if `grep` finds nothing, i.e. nothing is listening. Nmap then re-checks the port.

What you should observe: No listening socket on 8000, and Nmap usually reports `closed` (or another non-open state).

Check your understanding: Why might a scan report "filtered" rather than "closed" in a different environment?

Part C — Analysis and recording evidence (15–20 minutes)

Capture Firefox screenshots of the directory listing, the exposed file, the request details and the 404 verification. Include your Nmap output. You must find the filename through your own investigation.

Evidence item	Expected observation	Your screenshot / notes
E1 — initial HTTP test	GET / returns 200	
E2 — port discovery	8000/tcp open	
E3 — service identification	HTTP / Python service indication	
E4 — disclosure	GET to your discovered file returns 200 with fictional text	

Evidence item	Expected observation	Your screenshot / notes
E5 — retest	GET to the same URL returns 404	
E6 — shutdown	No service listening on port 8000	

Student vulnerability report template

- **Finding title:** Information disclosure via publicly accessible file.
- **Target and scope:** Record the exact URL YOU discovered under http://127.0.0.1:8000/.
- **Severity rationale:** Explain impact and limitations; do not assign a formal CVSS score without the required environmental details.
- **Description:** What configuration or file placement caused the problem?
- **Evidence before remediation:** Original discovered filename, URL, observed HTTP status and screenshot.
- **Potential impact in a real deployment:** What could be exposed?
- **Remediation:** Describe what was changed and why it works.
- **Verification:** Cite E5 and state whether the exposure was resolved.
- **Residual risk:** Explain that removing one file does not automatically secure an entire production website.

Part D — Theory questions

1. Define penetration testing. How does authorized testing differ from unauthorized probing?

2. Why is it useful not to know the vulnerable file's name before reconnaissance?

3. What is the purpose of the loopback IP address 127.0.0.1?

4. Differentiate a TCP port from the application service listening on it.

5. Explain the difference between `nmap -p 8000` and `nmap -sV -p 8000`.

6. What does HTTP 200 mean? Why does it not imply secure design?

7. Describe two kinds of metadata that HTTP headers may reveal.

8. Why can a publicly accessible file containing fictional account details illustrate a confidentiality weakness?

9. How did you demonstrate the exposure without using real accounts?

10. What changed when the exposed file was moved outside the web server directory?

11. Why should the same security test be repeated after applying a fix?

12. Why is Python http.server not suitable for hosting a production university portal?

Troubleshooting guide

Problem	Likely cause	What to run / do
Address already in use	Another process is listening on port 8000	`ss -ltnp | grep :8000` — stop your own process (Ctrl+C in its terminal), or use port 8081 consistently
Firefox: connection refused	Python server not running or wrong port	Check Terminal A is still running the server; confirm with `ss -ltn | grep :8000`
Nmap says closed	Server stopped or wrong address/port	Restart the server in Terminal A, then run `ss -ltn | grep :8000`
404 before the fix	Server started in the wrong folder	In Terminal A press Ctrl+C, run `cd ~/pentest-lab` and `pwd`, then restart the server
Nmap or curl: command not found	Package missing	`sudo apt update` then `sudo apt install -y nmap curl` (if package access is allowed)
Unexpected 200 after remediation	Another copy remains or wrong server root	`ls -la ~/pentest-lab` and `pwd` in Terminal A's folder; confirm the file was moved
Firefox headers differ from examples	Python / Firefox versions differ	Record your actual results, not illustrative output

Assessment rubric (100 points)

Criterion	Marks
Preparation and conceptual understanding	15
Correct single-machine setup and safe loopback binding	15
Port scanning and service identification	20
HTTP inspection and interpretation	15
Valid information-disclosure demonstration	15
Remediation and repeat verification	10
Clear evidence and final report	10
TOTAL	**100**

References / further reading

- Nmap Reference Guide — https://nmap.org/book/man.html
- Python documentation: http.server — https://docs.python.org/3/library/http.server.html
- Firefox Network Monitor documentation — https://firefox-source-docs.mozilla.org/devtools-user/network_monitor/
- OWASP Web Security Testing Guide — https://owasp.org/www-project-web-security-testing-guide/
- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment — https://csrc.nist.gov/pubs/sp/800/115/final